

Does a Music Transformer Know What Key It Is In?

Causal Interventions on an Emergent Tonal Representation

Anonymous ICASSP 2027 submission

Abstract—A small classifier can read the musical key from a music model’s activations, but reading a key does not show that the model uses it while it writes. We train Transformers on symbolic music whose vocabulary names pitches and timing, never a key. During generation we overwrite, in one layer, the 24 activation dimensions that carry the key with their average in another key, leaving every input note untouched. In a final test fixed in advance, on prompts no earlier run had seen, the continuation is in the key we install for 0.355 of the prompts, against 0.039 for the same write into random directions. Before the model generates anything, the probability it assigns to the notes of the new key already rises by +0.26, against +0.02, so the effect precedes feedback from its own output. Both results carry over to a public model trained on real music. A music Transformer therefore computes its key, holds it in a form we can overwrite, and consults it while it writes; we release the testbed.

Index Terms—music generation, interpretability, causal intervention, tonality, probing

I. INTRODUCTION

Music generation models behave as if they knew what key they were in [1], [2], [3]. A key is the small set of pitches that a passage treats as home: the music settles on them, leaves them, and comes back. Trained models reproduce this pattern over long spans, and they often move to another key and return. Their fluency raises one question that we can state precisely: does a model track the key somewhere inside, or does it only reproduce surface patterns of notes?

The evidence we have cannot answer that question, because it only shows that the key is readable. Classifiers can read the key out of the activations [4], and steering can push it, but neither shows that the model follows a key that we set (Section II). Game domains settle such questions by overwriting internal state; no study has run that test for musical key, and we supply it.

We settle that question by changing the key inside the model and looking at what it writes next (Fig. 1). At the core of our test is a single replacement: we take the part of one layer’s activation that carries the key and write in the value that this part takes in a different key, while every note of the input stays where it was, thereby changing the internal state of the model and nothing else. Musical key suits such a test better than most attributes do. First, music theory

defines a key outside the model [5], so we do not grade the model against a standard of our own making. Second, our vocabulary never names a key, so the key can only come from the pitches themselves. Third, the scale of a key is a fixed list of seven pitches, so we can check the edit by counting notes instead of training a judge to grade the music. The test therefore reads in one direction: if the music follows the value we install, the model was consulting that value; if it does not, we learn only that this value, at this layer and in this form, was not the one in use.

This paper is an analysis paper: we introduce no new generation method, and we claim no gain in generation quality. Our contributions are as follows. 1) We introduce a testbed for this question: a vocabulary that cannot leak the key, the true key on every note, and an evaluation protocol fixed in advance. 2) We show that the key is easier to read linearly from the activations than from counts of the notes, with controls for probe power and for untrained networks. 3) We show that overwriting the key moves the music into the key we write in—on prompts that no earlier experiment had seen, and also in a public model trained on real music. 4) We show that this effect is not an artifact: it appears in the next-note probabilities before the model writes anything, and it disappears when we restrict the edit to the positions where the model chooses pitches.

II. RELATED WORK

In this section we review two lines of work: music studies that read or push the key inside a model, and game studies that overwrite internal state. The second is the test we bring to music.

A. Probing and Steering in Music

Existing work on music models shows that the key is readable, not that the model uses it. Probing studies train a small classifier on the activations and read out music-theory concepts, from notes and chords to scales [4]; the key of a piece has been probed as well [6]. Steering studies use a technique adapted from language models [7], [8]: they add a learned vector to the activations, and an attribute of the output moves [9], [10], [11], [12]. Similar to our work, both act on the inside of the model, at inference time and without retraining. Probing and intervening have also been combined inside a music model, but with the intervention aimed at chord representations, not at installing a key for the model to follow [13]. Reading, however, does not mean using: a model can carry a signal inside and still ignore it when it writes [14], [15]. Steering cannot answer it either: an added vector leaves

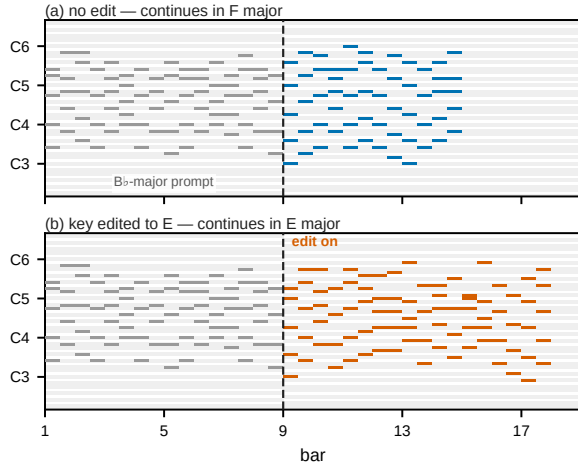


Fig. 1. The claim in one example (real model output, one fixed sampling seed). We continue the same 8-bar Bb-major prompt (a) without an edit and (b) after we write the E-major key part into the layer-4 activations from bar 9 on. The prompt is identical in both panels, and the continuation follows the key we install. In (a) the unedited continuation moves to the dominant, an ordinary move; in (b) it moves to E major, which is not. Shading marks the scale of the key in force.

the part of the activation that carries the current key in place, so pushing toward E major never sets the model’s key to E major, and no installed key exists for the model to follow. Our question sits one step further: write the key in, and watch whether the music follows.

B. Causal Tests by Overwriting

Game domains have already taken that step. Probing showed knowledge in games as well: it traced chess concepts inside AlphaZero [16]. Use, however, was settled by overwriting. A Transformer trained only to predict moves builds an Othello board inside itself [17], [18]. When researchers overwrite that board with a fake one, the model plays the moves that are legal on the fake board. Nothing in the input changed, so the new moves can only follow from the value that was written in: the model uses its board. Interventions of this type are called activation patching [19], [20], and the same test now covers chess-playing language models [21], mazes [22], and learned environment simulators in reinforcement learning [23]. In music the same name has also been used for additive steering [10], which adds a vector instead of replacing a value. Different from a game, where a move is legal or illegal at a glance, a musical continuation is never simply right or wrong. Our edit brings this patching test to music, and Section III adds the two tools music needs: a criterion for which key a continuation is in, and a limit on how much an edit may damage the music.

III. METHOD

In this section we present the two analyses of the paper, first reading the key and then overwriting it. As notation, write $h_\ell(t)$ for the activation at layer ℓ and position t , and $\kappa(t)$ for the true key there, one of 24 classes (12 tonics, each major or minor). In our own models, layers index the outputs of the eight Transformer blocks, 0 through 7: we never read from the raw token embedding. The probe distinguishes all

24 classes; the edits install the 12 major keys only (Sec. VI). Section IV describes the data, the models, and the evaluation protocol.

A. How We Read the Key

Our first question is whether the key is easier to read from $h_\ell(t)$ than from the notes themselves. The activation is computed from the notes, so the key is in there in some form by default; what we ask is whether the model has reshaped it into a form that is easy to read. We train a linear probe [24], [15] to predict $\kappa(t)$ from $h_\ell(t)$, and we split train and test by piece. A high probe score is not evidence on its own, because three things can produce one without the model computing anything about the key. First, a powerful probe can score points even on meaningless labels, so we shuffle the key labels within each piece and measure the probe on the shuffled labels [14]. That score is a floor the probe reaches regardless of content, and we subtract it from every probe-minus-baseline difference—every margin—that we report. Second, the activations of an untrained network might already score something, so we run the same probe on an untrained model. Third, if the notes themselves almost give the key away, then the model has added nothing. We therefore add a baseline that predicts the key from the notes alone: it counts how often each of the twelve pitch classes (C, C#, ..., B) occurs and guesses the key from those counts, by logistic regression and by Krumhansl-Schmuckler (KS) matching [25], [26] against the scale of each key, over windows of $W \in \{8, \dots, 64\}$ tokens. What the probe must beat, and by how much, is fixed in Section IV-C.

B. How We Overwrite the Key

Our second question, the main one, is whether the music follows the key when we overwrite it during generation. We prompt the model with 8 bars in a stable key, and from bar 9 on we edit one layer’s activations at every position it generates, following the activation-patching literature [19], [20]:

$$h \leftarrow h - P_V h + P_V \mu_{\kappa^*}, \quad (1)$$

where V collects the 24 directions that carry the key, $P_V = VV^\top$ keeps only the part of the activation that lies along them, and μ_{κ^*} is the average activation of the key we install. We orthonormalize the columns of V (by a singular value decomposition) before forming P_V , so P_V is an orthogonal projection. In plain terms, we swap the key-carrying part of the activation for the same part of a different key and touch nothing else—in particular, we never change the prompt. We build V in three ways: from the probe weights, from the per-key average activations, and from directions optimized for the edit itself [27]; the search of Section IV-D selects among them. The same edit also supports a generation-free reading: we append a key-neutral bar mark and beat position ourselves, apply the edit at the bar-9 boundary, and read the next-note probabilities without generating anything (Sec. V-C). We score this reading with two quantities, the generation-free versions of the measures of Section IV-C: the change in the log ratio between the probability mass that the next-note distribution puts on the installed key’s scale, in any octave, and the mass it puts on the prompt key’s scale, and the change in the mass on the installed scale itself.

Five comparisons bound the result. The first is a random-direction baseline: it writes into 24 randomly chosen directions instead of the key directions, so it changes the same number of directions inside the activation. This baseline does

not move the activation as far as the edit does: measuring the length of the change at each written position and averaging over positions, prompts, and target keys gives $\|\Delta\| = 34.6$ for the edit against 24.1 for the baseline, a ratio of 1.44, on activations of mean length 169. Our plan had asked the distances to match as well, the code matched only the number of directions, and we log that gap instead of hiding it. The second comparison therefore keeps the same 24 random directions and scales the change up to a distance of exactly 34.6, which removes the explanation that we simply pushed harder. The third is a do-nothing edit that must reproduce the unedited output token by token. The fourth applies one layer’s directions at a different layer, which tells us whether the directions belong to that depth. The fifth transposes the prompt itself, which tells us how far the output can move at all and so gives an upper bound. In sum, the probe tells us whether the key is there, and these five comparisons tell us whether the model uses it.

IV. EXPERIMENTS

In this section we describe what produces the results of Section V: the data, the models, what we measure, and the two-stage protocol, in that order.

A. Data

We need the true key at every note, so we generate the training data ourselves, following the synthetic, concept-isolating design of diagnostic datasets such as SynTheory [4]. A program that follows textbook harmony rules picks a key and writes each chord for four voices, optionally adding a melody. It produces 200k/10k/10k pieces for training, validation, and test, each 278–508 tokens long and containing 0–3 key changes. Because the program picked the key before writing, every token carries its true key, one of the 24 classes. The program draws keys near-uniformly: in the test split every major key occupies between 214k and 239k tokens (max over min, 1.12), so no key is rare. The model never sees these labels: they only train and score the probe. The vocabulary holds 124 symbols—bars, positions, absolute MIDI pitches, note lengths, and structural marks—and a unit test checks that no key, chord, or scale-degree symbol slips in.

B. Models

Our model is a GPT-2 style decoder [28], [29] with 8 layers, width 512, and about 25M parameters. We train it for next-token prediction, with and without transposition augmentation and with three random seeds each, which gives six models. A separate judge model shares the architecture but sees a different seed and a disjoint slice of the data. We never probe or edit the judge; it only rates whether edited music still looks like music, so no model grades its own output. All six models pass a quality gate that we set in advance: next-token top-1 accuracy of 0.878–0.881 on validation, against 0.434 for always guessing the most frequent token.

We also study one public model that we did not train: the Anticipatory Music Transformer [30], which its authors trained on Lakh MIDI [31], MetaMIDI [32], and FMA [33], and which we evaluate on 300 Bach chorales [1], [34] with human key labels [35]. We chose it because it is a public checkpoint in the symbolic domain trained on standard corpora, and the chorales with their published key analyses are a standard evaluation set. In that model we read the probe at the positions where it is about to choose a pitch, because its (time, duration, note) encoding forces that convention; the

supplementary shows that the verdict flips if we read after the note instead, where the probe no longer beats the note counts.

C. What We Measure

We fixed every measure and every rule for success before the corresponding run. For the probe, success means that the floor-corrected probe score beats the best note-counting baseline, with a bootstrap 95% confidence interval over pieces [36] that excludes zero. For the edit we use two measures and one limit. In every edited condition the model samples one continuation per prompt and target key, at temperature 1.0 with top- p 0.95 under a fixed seed, for at most 384 tokens, and we score its first 16 bars. The first measure is the success rate: how often the continuation, as judged by the KS estimator—the standard profile-matching key finder—is in exactly the key we installed. The estimator judges all the pitches of the scored continuation, and a continuation with fewer than 8 pitches counts as a failure. The second is the share of continuation notes that belong to a given scale, which needs no key estimator at all. The limit protects musical quality: before we ran a single edit, we measured how far the judge model’s perplexity rises across 7,989 natural key changes in the validation data—comparing the 24 tokens after each change with the 24 tokens before it—and fixed the 90th percentile of that rise, 0.613, as the most an edit may cost. At edit time we charge the limit on the whole continuation: the rise is the judge’s perplexity on the edited continuation minus its perplexity on the paired unedited one. For the public model we measure the same limit again from the natural key changes in the chorales, and we use a different public checkpoint as the judge. An edit may therefore disturb the music at most as much as a real key change does. A continuation that breaks the limit stays in the denominator and counts as a failure; we exclude nothing. A rate on its own means little, so we report every rate against the same write into random directions, which gives its floor.

D. Search Stage and Final Test

An intervention involves many choices, and picking them on the data that also produces the headline number inflates it, so we ran the study in two stages. The first stage compared 8 layers and 3 kinds of directions on 100 prompts and scored the best combination (probe directions at layer 4) on those same prompts, which the probe had also been trained on; we corrected for the 12 keys [37] but not for that search. We therefore report the first stage as a search, not as a test. The second stage draws 100 fresh prompts from a region of the test split that neither the probe nor any earlier run has seen. We wrote every such choice—the layer, the directions, the timing, the target keys, the baselines, the quality limit, the measures, the tests, the seeds, and the sample size—into a version-controlled document and committed it before the first number of this stage existed; we log every departure from it and report the results that went against us. Throughout the paper, the tests are one-sided Wilcoxon signed-rank tests for each target key, paired by prompt and corrected across the 12 keys with Holm’s method [37], and the effect size r is the rank-biserial correlation. We repeat no search over layers or directions. Because Section V-E shows what an imbalanced estimate of the targets can fabricate, we also checked the targets here: every major key contributes between 7,530 and 9,237 positions to its class mean (max over min, 1.23), and no mean is degenerate. We leave out edits whose target key

equals the key of the prompt and report them separately as a sanity check. From this point on the design could only pass or fail, and nothing in it was left to tune toward the outcome we report.

V. RESULTS

In this section, we take the claims of Section I in order; headline numbers come from the transposition-augmented model with seed 0, and a second augmented seed replicates every search-stage verdict. We did not run the final test on that second seed: its do-nothing edit failed to reproduce the unedited output on 1 of the 100 prompts, an artifact of floating-point summation order, so the seed did not meet a precondition of the frozen design.

A. Reading the Key

The model works out the key; it does not copy it off the surface of the input. At layer 4 the probe reaches macro- F_1 0.927, the best note-counting baseline reaches 0.790, and the shuffled-label floor is 0.032, so the corrected margin is +0.105 (bootstrap 95% CI [0.091, 0.114]). This margin stays above zero in layers 1–7 of all six trained models, so the rule we set in advance holds in 6/6 cases. It turns negative only at layer 0, the first block’s output and the reading point closest to the input (−0.20), and an untrained model reaches only 0.243.

A longer history does not close this gap. The note-counting baseline above sees at most 64 tokens, so we extended it in three ways (analyses added after our original protocol, which had fixed $W \leq 64$, and before the final-test freeze): to the whole prefix, to a recency-weighted count, and to a count that joins a short and a long window. Widening the window alone peaks at 0.801 for $W=96$ and falls to 0.752 at $W=512$, because notes from before a key change pull the count the wrong way; the strongest of the three variants, the one that joins a short and a long window, reaches 0.824. Against that strongest variant the margin is still +0.071 [0.056, 0.081], so we claim that the activation makes the key easier to read linearly than these note counts do—not that it beats every function of the notes.

B. Overwriting the Key

Overwriting the key moves the music into the key we install, and a random overwrite does not. On the 100 fresh prompts of the final test (Fig. 2), the edit reaches a success rate of 0.355 against 0.039 for the random baseline, a ratio of 9.1. All 12 target keys separate after correction (with $p = 4.8 \times 10^{-5}$, effect size $r = 0.72$ –1.00), and the prompt-by-prompt difference has a bootstrap 95% CI of [0.284, 0.347]. The quality limit admits 84% of edits, the same rate without that limit is 0.410, and the notes cross over as well (0.936 of them in the installed key against 0.617 in the prompt’s). The length-matched random baseline of Section III-B reaches 0.056, and the edit separates from it in 12/12 keys after correction. Both random baselines also pass the quality limit more often than the edit does (0.96 and 0.93 against 0.84), and without that limit they still reach only 0.041 and 0.059. The effect therefore follows from which directions we write to, and not from how far we push the activation or from a limit that penalizes the baselines. When we install the key the prompt is already in, the model simply stays there (0.650 scored as the headline rate, 0.710 before the quality limit, against the 51.7% of continuations—also before the limit—that stay in the prompt’s key under the random edit below),

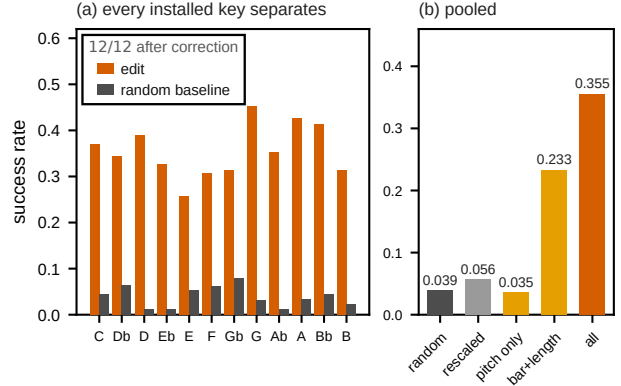


Fig. 2. The main causal result, on the fresh prompts of the final test. (a) Success rate for each key we install, edit against the random baseline that changes the same number of directions. (b) The same measure pooled, for the edit at all positions and its four comparisons: random directions; the same random directions rescaled to the size of the edit; the write at pitch positions only; and the write at bar and note-length positions only.

and the do-nothing edit reproduces the unedited output token by token (100/100), as both should.

Specificity. The edit does not merely leave the old key; it lands on the new one. On the fresh prompts, scored before the quality limit turns violations into failures and over the continuations whose key the estimator can judge, the estimated key of the continuation matches the key we install 41.0% of the time, against 4.1% for the random baseline, while only 8.5% of continuations stay in the key of the prompt, against 51.7%. Most of what remains falls on the neighbor a fifth away (34.8%), which the KS estimator is known to confuse. The edit therefore installs one specific key rather than merely disrupting the current one.

C. The Next Note, Before Any Feedback

The edit changes the model’s next decision before any note it wrote can feed back. A sustained edit leaves one alternative explanation: the edit may disturb the first few notes, and the model may then read the key back off its own output. We rule that out with the generation-free reading of Section III-B, scored with the two quantities defined there. The edit raises the log ratio toward the installed key by 0.695—it roughly doubles the ratio—while the random baseline raises it by 0.041. The mass on the installed scale rises by 0.255 from 0.547 without an edit, against 0.017 for it. The separation holds in 12/12 keys after correction, with a worst case of $p = 5 \times 10^{-16}$ and an effect size of $r \geq 0.99$. The push in the probabilities is therefore already there before a single edited note exists.

The effect persists through the notes, not through a memory inside the model. In the search stage, a single write kept the notes in the new key for many bars. Yet splicing in the tokens of that one bar, with no edit in force, reproduces the whole effect. The model therefore reads the key back off the notes at every step and keeps no memory of the key apart from what it has written.

D. Where the Key Is Read and Where It Acts

The layers where we can read the key are not the layers where the model uses it. The probe is flat across a wide band

(macro- F_1 0.920–0.929 for layers 2–6), yet editing each layer in turn and subtracting the random baseline leaves a single peak in the middle of the network (best against median, CI [0.038, 0.109]): at layers 0 and 1 the probe already reads the key well and the edit does almost nothing (Fig. 3). These are search-stage data and we did not repeat the sweep on the fresh prompts of the final test, so we report the layer profile as an observation rather than as a tested claim.

Two further search-stage comparisons bound the effect. First, one layer’s directions still worked when we wrote them at another layer: the directions of layers 5–7, written four layers earlier at layers 1–3, reached success rates of 0.27–0.39, while the writes in the other direction stayed at the random-baseline floor (0.003–0.082). This does not contradict the narrow causal band: transfer asks whether the directions carry across depths, while the band asks where writing them changes the output. Second, the edit covered 58% of the distance that the output moves when we transpose the prompt itself, and that transposition gives the upper bound. Section V-C suggests why the edit stays below that bound: the model re-reads the key off the notes at every step, so the notes of the prompt, which we never touch, keep pulling the continuation back toward the old key. Like the layer profile, we report both comparisons as observations, not as tested claims.

The positions we write to matter as much as the depth. Writing only at pitch positions does nothing (success rate 0.077, at the random-baseline floor of 0.075) and leaves the music intact, while writing only at bar and note-length positions carries 69% of the full effect. The final test reproduces both halves: writing only at pitch positions reaches 0.035, below the random-baseline floor of 0.039, and separates from it in 0/12 keys, while writing only at bar and note-length positions reaches 0.233, or 65% of the full effect, and separates in 11/12. Writing only at pitch positions does keep more continuations inside the quality limit (98% against 84%), but only because it barely changes the music. An intervention that merely pushed on pitch probabilities would behave the other way around. Reading a value linearly is therefore a weak guide to where that value is used, which matches the report from language models that the direction easiest to read is not the one that steers best [38], [39]. The edit fails exactly where the key is easiest to read.

E. A Public Model

Both results carry over to the public model of Section IV. The probe beats the note-counting baselines by +0.196 [0.132, 0.260].

The edit transfers too. It reaches $7.8\times$ the random baseline (0.479 against 0.061), separates all 12 targets, keeps 99.7% of continuations inside the limit, and moves the notes across as well (0.890 of them in the installed key against 0.753 in the prompt’s). Here too the layer that acts (layer 8) is not the layer that reads best (layer 10).

One first-run finding did not survive its control and we withdraw it: what looked like a learned preference against $F\sharp$ major came from estimating the targets on a corpus in which that key never occurs, and the supplementary reports the corrected numbers.

VI. LIMITATIONS

We state what this study does not claim. We claim a causally used internal representation of key, not a human-like concept of it: the model keeps no memory of the key

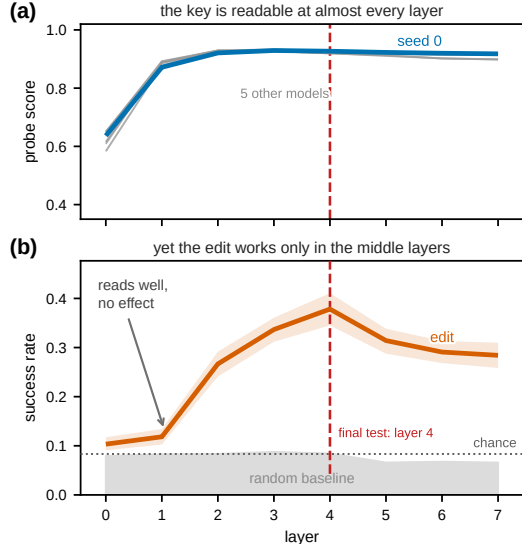


Fig. 3. (a) The probe score per layer—the macro- F_1 of Section V-A—for all six models (seed 0 highlighted). We can read the key across a broad band. (b) Success rate when we edit at each layer (band: prompt-level bootstrap 95% CI); the gray region reaches up to the random baseline’s confidence ceiling, and the dotted line marks chance (one key in 12). The output changes only in a narrow band. Search-stage data; the final test fixes layer 4.

apart from the notes it has written (Sec. V-C), and our test speaks to use, not to understanding. We install major keys only; we registered minor keys as a secondary condition and have not run them. The KS estimator confuses keys a fifth apart, which is why we also report the share of notes that belong to the scale. Our quality limit measures how far an edit disturbs a reference model, not musical quality itself, and we ran no listening test, although in the symbolic domain the notes themselves let us check scale membership objectively. The public-model result covers one checkpoint at one size, and it keeps its original design, in which we chose the layer on separate prompts and fixed the limit before evaluation but did not run a second final test. The supplementary reports our scaling and geometry analyses. We leave minor keys and a listening test for future work.

VII. CONCLUSION

We asked whether a music Transformer knows what key it is in, and overwriting settles it: the music follows the key we write in. Readers may call the thing we edit a world model; we use that term as an interpretation, not as a result. We release the testbed, the documents that fix the design, and demos: <https://anonymous.4open.science/r/TBD>.

REFERENCES

- [1] G. Hadjeres, F. Pachet, and F. Nielsen, “DeepBach: A steerable model for Bach chorales generation,” in *Proc. International Conference on Machine Learning*, 2017.
- [2] C.-Z. A. Huang *et al.*, “Music Transformer: Generating music with long-term structure,” in *Proc. International Conference on Learning Representations*, 2019.
- [3] J. Copet *et al.*, “Simple and controllable music generation,” in *Proc. Advances in Neural Information Processing Systems*, 2023.

- [4] M. Wei, M. Freeman, C. Donahue, and C. Sun, “Do music generation models encode music theory?” in *Proc. International Society for Music Information Retrieval Conference*, 2024.
- [5] C. L. Krumhansl and E. J. Kessler, “Tracing the dynamic changes in perceived tonal organization in a spatial representation of musical keys,” *Psychological Review*, vol. 89, no. 4, pp. 334–368, 1982.
- [6] R. Castellon, C. Donahue, and P. Liang, “Codified audio language modeling learns useful representations for music information retrieval,” in *Proc. International Society for Music Information Retrieval Conference*, 2021.
- [7] K. Li, O. Patel, F. Viégas, H. Pfister, and M. Wattenberg, “Inference-time intervention: Eliciting truthful answers from a language model,” in *Proc. Advances in Neural Information Processing Systems*, 2023.
- [8] A. Zou *et al.*, “Representation engineering: A top-down approach to AI transparency,” *arXiv preprint arXiv:2310.01405*, 2023.
- [9] J. Koo, G. Wichern, F. G. Germain, S. Khurana, and J. Le Roux, “SMITIN: Self-monitored inference-time intervention for generative music transformers,” *IEEE Open Journal of Signal Processing*, vol. 6, pp. 266–275, 2025.
- [10] S. Facciano, G. Strano, D. Crisostomi, I. Tallini, T. Mencattini, F. Galasso, and E. Rodolà, “Activation patching for interpretable steering in music generation,” *arXiv preprint arXiv:2504.04479*, 2025.
- [11] N. Singh, M. Cherep, and P. Maes, “Discovering and steering interpretable concepts in large generative music models,” in *Proc. International Conference on Learning Representations*, 2026.
- [12] D. Zhao, D. Beaglehole, T. Berg-Kirkpatrick, J. McAuley, and Z. Novack, “Steering autoregressive music generation with recursive feature machines,” in *Proc. International Conference on Learning Representations*, 2026.
- [13] W. Ma and G. Xia, “Exploring the internal mechanisms of music LLMs: A study of root and quality via probing and intervention techniques,” in *Proc. ICML Workshop on Mechanistic Interpretability*, 2024.
- [14] J. Hewitt and P. Liang, “Designing and interpreting probes with control tasks,” in *Proc. Conference on Empirical Methods in Natural Language Processing*, 2019.
- [15] Y. Belinkov, “Probing classifiers: Promises, shortcomings, and advances,” *Computational Linguistics*, vol. 48, no. 1, pp. 207–219, 2022.
- [16] T. McGrath, A. Kapishnikov, N. Tomašev, A. Pearce, M. Wattenberg, D. Hassabis, B. Kim, U. Paquet, and V. Kramnik, “Acquisition of chess knowledge in AlphaZero,” *Proceedings of the National Academy of Sciences*, vol. 119, no. 47, 2022.
- [17] K. Li, A. K. Hopkins, D. Bau, F. Viégas, H. Pfister, and M. Wattenberg, “Emergent world representations: Exploring a sequence model trained on a synthetic task,” in *Proc. International Conference on Learning Representations*, 2023.
- [18] N. Nanda, A. Lee, and M. Wattenberg, “Emergent linear representations in world models of self-supervised sequence models,” in *Proc. BlackboxNLP Workshop on Analyzing and Interpreting Neural Networks for NLP*, 2023.
- [19] A. Geiger, H. Lu, T. Icard, and C. Potts, “Causal abstractions of neural networks,” in *Proc. Advances in Neural Information Processing Systems*, 2021.
- [20] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, “Locating and editing factual associations in GPT,” in *Proc. Advances in Neural Information Processing Systems*, 2022.
- [21] A. Karvonen, “Emergent world models and latent variable estimation in chess-playing language models,” in *Proc. Conference on Language Modeling*, 2024.
- [22] A. F. Spies, W. Edwards, M. I. Ivanitskiy, A. Skapars, T. Räuker, K. Inoue, A. Russo, and M. Shanahan, “Transformers use causal world models in maze-solving tasks,” *arXiv preprint arXiv:2412.11867*, 2024.
- [23] X. Zhang, “What do world models learn in RL? probing latent representations in learned environment simulators,” *arXiv preprint arXiv:2603.21546*, 2026.
- [24] G. Alain and Y. Bengio, “Understanding intermediate layers using linear classifier probes,” in *Proc. International Conference on Learning Representations, Workshop Track*, 2017.
- [25] C. L. Krumhansl, *Cognitive Foundations of Musical Pitch*. Oxford University Press, 1990.
- [26] D. Temperley, “What’s the key for a piece of music? The Krumhansl–Schmuckler key-finding algorithm reconsidered,” *Music Perception*, vol. 17, no. 1, pp. 65–100, 1999.
- [27] A. Geiger, Z. Wu, C. Potts, T. Icard, and N. D. Goodman, “Finding alignments between interpretable causal variables and distributed neural representations,” in *Proc. Conference on Causal Learning and Reasoning*, 2024.
- [28] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Proc. Advances in Neural Information Processing Systems*, 2017.
- [29] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” *OpenAI Technical Report*, 2019.
- [30] J. Thickstun, D. Hall, C. Donahue, and P. Liang, “Anticipatory music transformer,” *Transactions on Machine Learning Research*, 2024.
- [31] C. Raffel, “Learning-based methods for comparing sequences, with applications to audio-to-MIDI alignment and matching,” Ph.D. dissertation, Columbia University, 2016.
- [32] J. Ens and P. Pasquier, “Building the MetaMIDI dataset: Linking symbolic and audio musical data,” in *Proc. International Society for Music Information Retrieval Conference*, 2021.
- [33] M. Defferrard, K. Benzi, P. Vandergheynst, and X. Bresson, “FMA: A dataset for music analysis,” in *Proc. International Society for Music Information Retrieval Conference*, 2017.
- [34] C.-Z. A. Huang, T. Cooijmans, A. Roberts, A. Courville, and D. Eck, “Counterpoint by convolution,” in *Proc. International Society for Music Information Retrieval Conference*, 2017.
- [35] M. Gotham, G. Micchi, N. N. López, and M. Sailor, “When in Rome: A meta-corpus of functional harmony,” *Transactions of the International Society for Music Information Retrieval*, 2023.
- [36] B. Efron, “Better bootstrap confidence intervals,” *Journal of the American Statistical Association*, vol. 82, no. 397, pp. 171–185, 1987.
- [37] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [38] Z. Wu, A. Arora, A. Geiger, Z. Wang, J. Huang, D. Jurafsky, C. D. Manning, and C. Potts, “AxBench: Steering LLMs? even simple baselines outperform sparse autoencoders,” in *Proc. International Conference on Machine Learning*, 2025, pp. 67 035–67 080.
- [39] D. Arad, A. Mueller, and Y. Belinkov, “SAEs are good for steering—if you select the right features,” in *Proc. Conference on Empirical Methods in Natural Language Processing*, 2025.